COMS4060A/7056A Assignment 1: Exploring and Cleaning a Fuel Logbook Dataset

Group Members

Name	Surname	Student Number
Sinethemba	Nani	1607717
Nokukhanya	Nkosi	2192323
Dolf	Shungube	2683067
Mashego	Mabeloane	2654606

Contents

Preprint.

1 Data Cleaning

1.1 Date Fields

1.1.1 Percentage of invalid `date_fueled` entries

The data_fueled column has 88.3% entries that are valid dates and the remaining 11.7% are not proper dates. These values were obtained by trying to convert each entry of the date_fueled column to a date-time entry and all that failed are seen as improper dates.

1.1.2 Using `date_captured` as a proxy

For most entries in the data-set date_fueled entries are the same as their corresponding date_captured entry so we have decided that if one of the values has a NaT value, we can replace it with its corresponding partners value assuming it is not null. This is valid because most users would capture this record in the same date they fueled their vehicle to avoid forgetting.

1.1.3 Converting to date format

To set the dates to a proper date format we converted each entry to a date-time entry, and pandas has a function for this action. Specifically only entries with the format "Month Date Year" were converted to proper date-time entries. This function automatically sets values without the set format into NaT entries.

1.1.4 Removing implausible dates

As we are working with date-time entries in all date columns, we are able to use the [dt.year.isin([2005])] to extract all the dates with a year value of 2005. The year 2005 entries form 0.0111% of the total data in the full data-set.

1.1.5 Distribution of fueling dates

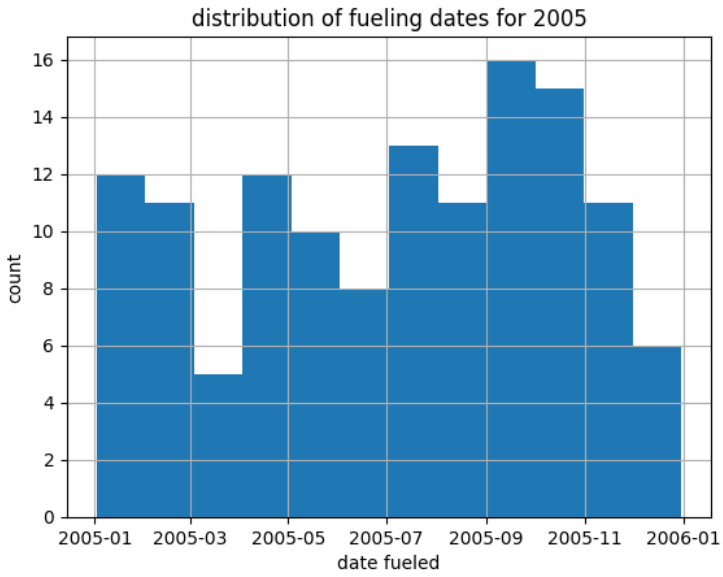

Figure 1: Distribution of `date_fueled` values after cleaning.

The figure shows that most months had around 12 fueling entries recorded in the app for the year 2005, the highest value being 16 recorded fueling entries between September and October, these small values suggest that during 2005 the app was not popular, and this is further supported by the fact that the year 2005 entries only form 0.0111% of the full data-set.

1.2 Numeric Fields

1.2.1 Percentage missing: gallons, miles, odometer

The gallons column initially had 11.54% missing entries, and this value remained the same after trying to calculating some of its missing values.

The miles column initially had 87.55% missing entries and after calculating some of the missing values, this dropped to 73.94%.

The odometer column had 12.69% missing entries, and since it is not possible to deduce the value of the missing entries, this value remained as is.

The mpg column initially had 84.33% missing entries and after calculating some of the missing values, this dropped to 74.65%.

1.2.2 Using interdependence to impute

When only the miles value was missing, with gallons and mpg present then its value was calculated as $(mpg \times gallons)$.

When only the gallons value was missing, with miles and mpg present then its value was calculated as $\frac{miles}{mpg}$.

When only the mpg value was missing, with miles and gallons present then its value was calculated as $\frac{miles}{gallons}$.

Note that for all the calculations that used division, we got a value of ∞ if we devided by zero , so after the calculations any entry for gallons or mpg with ∞ was set to NaN.

1.2.3 Converting strings to floats

If entries had thousands-separator commas, these commas were removed by replacing them with the empty-character before trying to convert the value to a float. we used the str.replace() function for this action.

After removing the commas from all entries, the values could be converted to floats using the pandas to_numeric() function. all entries that failed to be converted to a float were set to NaN.

1.2.4 Plotting the distributions

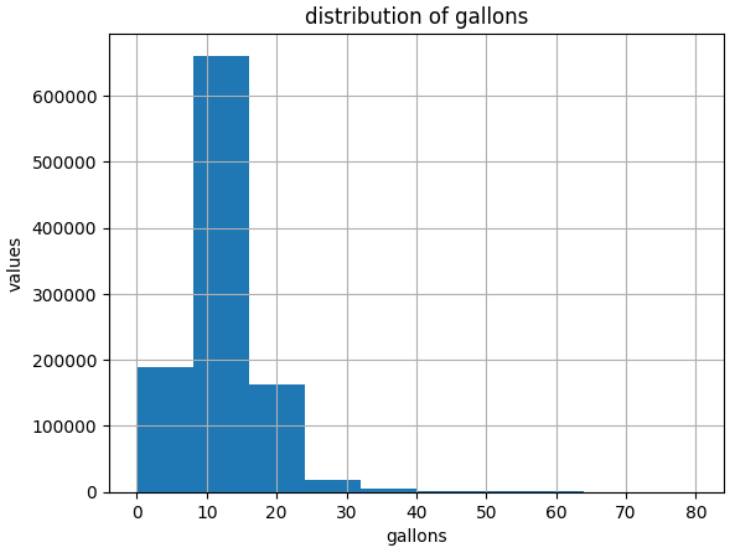

Figure 2: Distributions of gallons after cleaning.

To prepare this plot most of the extremely high gallon entries were removed, this was done from the assumption that we are working to normal commercial vehicles, our research showed that most regular sized cars can hold at-most 20 gallons of fuel and this can go to 40 gallons for larger vehicles so we limited our data to include up to 80 gallons.

This assumption is further supported by the distribution which shows that most entries get about 9 to 15 gallons of fuels. The value being lower than the expected is because most individuals will not go to fuel up when they have exactly 0 gallons of fuel.

The plot also shows that the entries with greater that 40 gallons of fuel are more scattered.

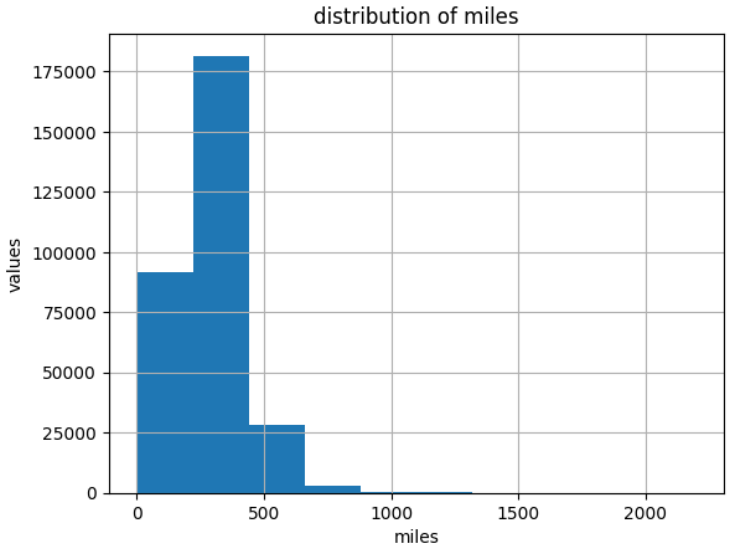

Figure 3: Distributions of miles after cleaning.

To prepare this plot, the miles entries with extremely high value we removed, this was done because it unrealistic to think a normal car could travel extreme distance from a single tank of fuel,so we wanted to deal with the realistic values only thus limited the miles values to 2500 miles

The removal of the extreme value is further supported by the fact that the world record for the longest distance traveled with a single full tank with a normal passenger car is 1,759 miles achieved in 2025 achieved by Miko Marczyk.

The plot supports this observation, with most entries being 250-500 miles traveled from the single fuel-up and the count dropping with an increase in miles and values above 1500 miles being more spread out. The values from 0-250 miles most-likely show vehicles with smaller fuel tanks or those who received small amounts of fuel.

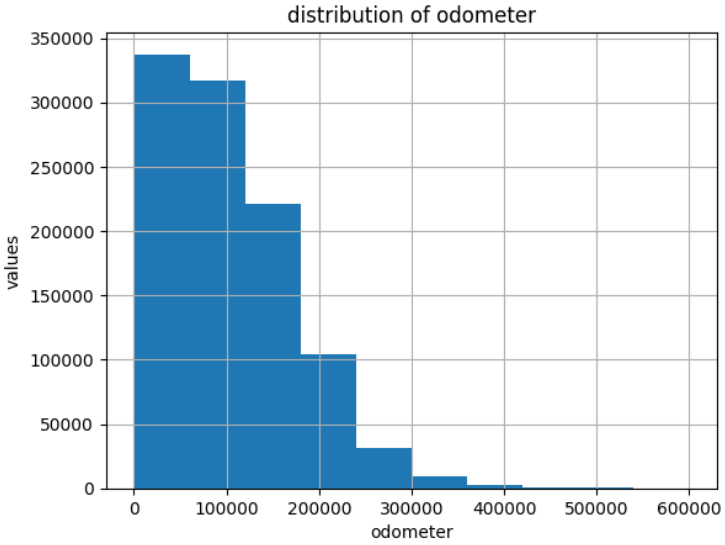

Figure 4: Distributions of odometer after cleaning.

To prepare this plot, we only took odometer readings whose entry was not above 600 000. This was done because most cars last for around 500 000 miles, with extensive maintenance and repairs.

600 000 being a great cutoff is supported by the fact that that after 500 000 entries are more scattered.

The plot shows that most vehicle have an odometer reading below 100 000 miles, with the number of vehicles in each odometer group decreasing with an increase in odometer reading which is an expected pattern as it can become harder to maintain older vehicles thus there should be fewer vehicles with high odometer readings.

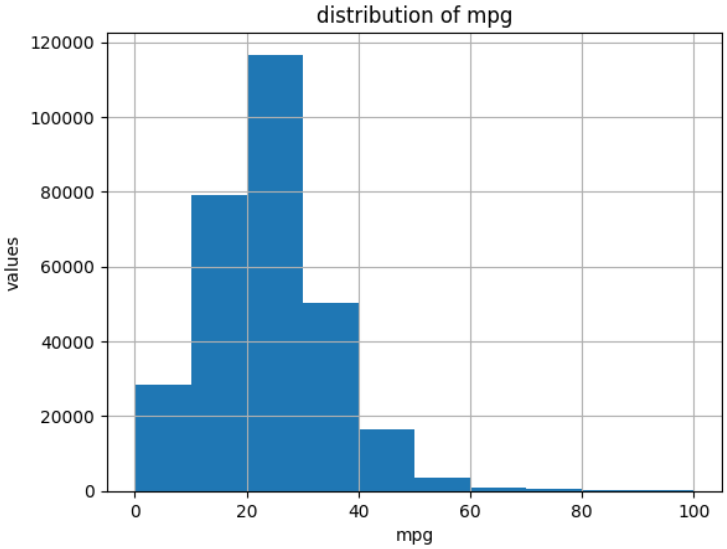

Figure 5: Distributions of mpg after cleaning.

To prepare this plot mpg entries above 100 mpg were removed, this was done because most vehicle mpg rating are far below this value, this assumption is further supported by the fact that the highest recorded mpg value for normal passenger vehicles is 93.158 mpg set with a Toyota Prius in 2024.

The plot shows that most mpg readings are between 20 and 30 which is expected for normal passenger vehicles, with fewer vehicles having values above this, this further supports that it is unlikely for a passenger vehicle to have an mpg reading above 100 mpg.

The data-cleaning process used to achieve these plots removed 0.0017244 of the original data.

1.2.5 Statistical description

Table 1: Summary statistics for numeric fields.

Column	Mean	Std	Min	Max	Mode	Quartiles
gallons	12.31774	5.4382	0	80	10.567	[8.99375, 11.954, 14.93525]
miles	277.987	144.334	0	2198.259	0	[201.5831,277,351.625]
odometer	102798.7417	72169.8514	0	599926.0	1.0	[45913.0,91846.0,146832.0]

The Max value of each column is greatly affected by the choice of value-cutoff that was chosen during the data-cleaning process,this does add some bias but these values were specifically chosen to stay within a more realistic range of these entries.

The mean value for gallons shows that fueling entries tend to be around 12.31774 gallons, this is expected because most passenger vehicles hold about 20 gallons of fuel and people would be fueling up around this range to avoid their car being out of fuel. This is further supported by the fact that the mode is also close to this value, so most people fuel-up when their tank is about half-empty. The Std being small shows that there isn't much of a variance in the amount of fuel used during fuel-up.

The mean miles suggest that the average travel distance per single fuel-up is 277.987 miles which should be expected as most people will not be traveling large distances. The mode being at 0 is a bit suspicious as it suggests that most people never use their car, which is highly unlikely especially after the fueled up the car The large std is most likely a result of the Max value value and the the small minimum.

The mean odometer reading suggests that cars have an average reading of 102798.7417 which is expected but the mode being 1.0 is a bit suspicious as it suggest that most cars have never been used which is extremely unlikely. The high std is a result of the great difference between the minimum and the Maximum combined with.

2 Feature Engineering

2.1 Currency column

Currency was extracted with a non-greedy regular expression applied to `cost_per_gallon`:

df['currency'] = df['cost_per_gallon'].str.extract(r'(.*?)\d+')

The pattern `(.*?)\d+` captures every character from the start of the string up to, but not including, the first digit encountered. Because the group is non-greedy, it stops at the earliest possible digit, which correctly isolates the currency marker regardless of whether it is a single symbol or multiple-character code:

- Single-character symbols like ($, £, €) are captured directly, e.g. `"$5.599"` → `"$"`
- Three letter ISO-style with no separator, such as the Roman leu, are captured in full because none of their characters are digits, e.g. `"RON16.69"` → `"RON"`.
- **Edge case Swiss Franc:** entries for CHF are written with a trailing period before the amount, e.g. `"CHF.16.69"`. Since the regex only halts at the first digit, the period is swept into the captured group along with the letters, producing `"CHF."` rather than `"CHF"`.This is a direct consequence of the non-greedy match rather than a parsing error, but it means the resulting `currency` column is not fully normalised, the same underlying currency can appear with or without a trailing period depending on how the source field was formatted, and any downstream equality check against a currency label must account for this exact string.

Currency was extracted only once, from `cost_per_gallon`; `total_spent` was assumed to share the same currency for each row and was not parsed separately

2.2 Float value of total spend and cost per gallon

Both currency-prefixed field were converted to numeric valued with a shared helper, `to_float`, which:

- Returns `NaN` immediately for missing values.
- Searches the string for the first numeric token using `\d[\d,]*\.?\d*`, which matches a leading digit followed by any mix of further digits and thousands-separator commas, an optional decimal point, and trailing digits.
- Strips the commas from the matched token and casts the result to `float`.

This discards the leading currency marker (symbol or code) entirely and applies uniformly to `cost_per_gallon` and `total_spent`, producing `cost_per_gallon_value` and `total_spent_value` as clean numeric columns.

2.3 Car make, model, year, and User ID from URL

Each `user_url` was passsed with `urllib.parse.urlparse`, taking only the path component and splitting it on `/`, with empty segments (from leading/trailing slashes) filtered out. The remaining path segments were read from right to left, since the URL structure places the most identifier last:

- Last segment → `user_id`
- Second-to-last segment → `year`
- Third-to-last segment → `model`
- Fourth-to-last segment → `make`

Rows with a missing URL, or with fewer than the required number of path segments, yield `NaN` for the corresponding field(s) rather than raising an error.

2.4 Metric Unit Conversions

2.4.1 Litres filled

The source data records fuel volume in gallons, an imperial unit. South Africa uses the metric system, so `gallons` was converted to `litres_filled` rather than left in its original unit.

A single gallon-to-litre constant is not sufficient, however, because a UK (imperial) gallon and a US (liquid) gallon are different volumes:

- UK (imperial) gallon $= 4.54609$ L
- US (liquid) gallon $= 3.785411784$ L

2.4.2 Kilometres driven

`km_driven` was computed by scaling `miles` by the standard mile-to-kilometre conversion factor:

df['km_driven'] = df['miles'] * 1.60934

This is roughly a 20% difference, so using the wrong constant materially over or under-states `litres_filled`, and that error then propagates into `litres_per_100km`. The dataset has no explicit country field, so `currency` was used as a proxy for locale when deciding which constant applies to each row:

- **£ (pound) - UK gallon.** The pound is the one currency in the dataset that specifically signals the UK, a country where the gallon remains in colloquial fuel-volume use, so pound-denominated records use `UK_GALLON_L`.
- **Every other currency ($, €, RON, CHF.) - US gallon**, applied as a default rather than a strong locale claim. €, RON, and CHF correspond to countries (euro-zone, Romania, Switzerland) that natively measure fuel in litres, not gallons, so a gallon reading for those rows is already an artefact of the dataset rather than a locale-accurate unit; the US gallon was chosen as the more commonly referenced default definition of "gallon" in the absence of better information. The $ symbol is itself ambiguous (USD, CAD, AUD, etc.), so treating it as US-denominated is an assumption rather than a guarantee.

2.4.3 Litres per 100km

Fuel efficiency was expressed in litres per 100 kilometres, the standard metric consumption measure, by dividing litres filled by kilometres driven and scaling to a 100 km basis:

df['litres_per_100km'] = df['litres_filled'] / df['km_driven'] * 100

This value is only defined where `miles` is present, so rows lacking `miles` return `NaN` for `litres_per_100km` even when `litres_filled` was successfully computed.

3 Vehicle Exploration

3.1 Unique users per country

Figure 6: Number of unique users per country (proxied by currency).

To create this plot, the countries were divided into 4 groups and each group plotted separately so that the plot is easy to see. The plot is too large to include in this document, see the corresponding notebook.

This plot shows that the largest group of users is from the US with almost 80 000 unique users. The lowest number of users is 1, which is a tie between L$,KGS, CUS, Af, KZT,YR and TMT.

3.2 Unique users per day

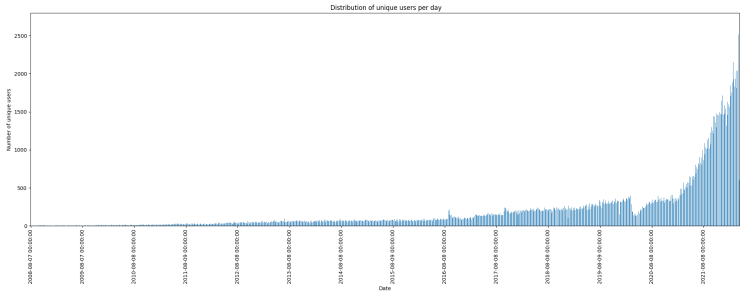

Figure 7: Number of unique users per day.

This plot shows an increase in the number of users per-day with an increase in time, which shows in increase in the popularity of the app overtime maxing out at 2500 unique users in a single date.

3.3 Distribution of vehicle age per country

Figure 8: Distribution of vehicle age per country.

This plot shows a separate distribution for each country which makes it very large thus would be impractical to attach to the document. See the notebook for the full plot.

The plots show that in most countries the vehicles are mostly around 10 years old and few cars make it above 40 years old.

3.4 Most popular makes and models

The plot for model divides the entries into 35 groups and because of this, the plot it too large to include in this document, see the notebook.

The most popular model is the civic with around 8000 users and the most popular make is ford with around 140 000 users.

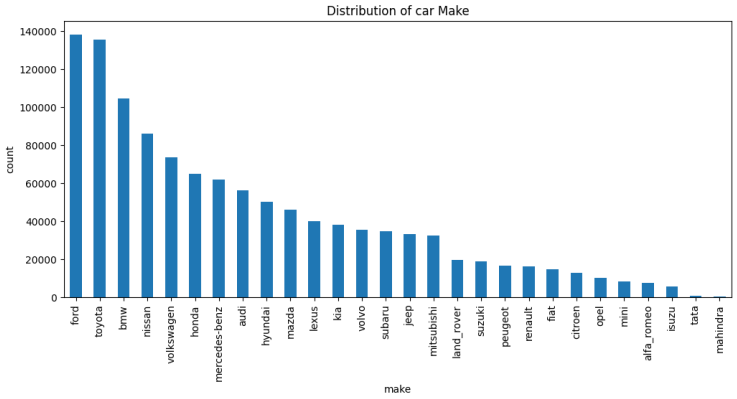

Figure 9: Distribution of vehicle age per country.

4 Fuel Usage

4.1 Outlier Removal

4.1.1 Top 5 currencies by number of transactions

We restrict outlier detection to the five most frequent currencies in the dataset, since fuel-purchase behaviour (and therefore what counts as an outlier) is fundamentally different across regions. Table 2 shows the five currencies with the highest transaction counts.

Table 2: Top 5 currencies by transaction count.

Currency	Transactions
$ (USD)	56,910
£ (GBP)	6,619
€ (EUR)	4,485
CA$ (CAD)	3,548
R (ZAR)	2,785

4.1.2 Removing outliers per currency

Outlier removal was carried out in three stages, applied separately to each of the five currencies above, since a normal transaction in Rands looks nothing like a normal transaction in Dollars.

Layer 1 – known currency-mislabelling errors. As flagged in the assignment brief, some South African users have their currency incorrectly recorded as $ rather than R. These rows are identifiable because the implied cost per litre is far too high to be a genuine US fuel price. We remove any $-currency transaction with `cost_per_litre` $>$ $5.00, which caught 1,995 rows.

Layer 2 – structural inconsistency. For every transaction we compare the recorded `total_spent` against `gallons` $\times$ `cost_per_gallon`. Where these disagree by more than 50%, the entry is internally inconsistent (e.g. a scraping or unit error) regardless of currency or magnitude, and is removed. This removed a further 1 row.

Layer 3 – statistical and physical bounds. On the remaining data, for each currency separately, we compute Tukey bounds ($Q_1 - 1.5\,\mathrm{IQR}$, $Q_3 + 1.5\,\mathrm{IQR}$) on `total_spent`, `litres_filled`, `cost_per_litre`, `gallons`, and `litres_per_100km`, and intersect these with physically plausible hard limits (e.g. 2–50 L/100km for fuel efficiency, per the brief's own sanity-check guidance). A row is removed if any of these five fields falls outside its bound; missing values in a field are not treated as violations, since a gap in one field should not disqualify an otherwise valid transaction. Table 3 shows the resulting per-currency bounds.

Table 3: Layer-3 bounds by currency, after Layers 1–2.

Currency	Cost/L (low–high)	Gallons (low–high)	Litres filled (low–high)	L/100km (low–high)	Total spent (low–high)
$	0.28–1.47	1.00–23.90	3.80–90.48	2.00–20.93	1–89.94
CA$	0.68–1.92	0.40–23.01	1.50–87.10	2.00–19.93	1–118.23
R	6.77–22.34	0.10–27.23	0.50–103.09	2.00–18.26	1–1596.58
£	0.72–1.38	0.10–22.72	0.50–103.28	2.00–17.23	1–110.83
€	0.76–2.01	0.10–23.16	0.50–87.66	2.00–13.56	1–120.54

These bounds are consistent with real-world fuel prices over the period covered by the dataset – e.g. roughly R6.77–22.34 per litre for South Africa versus $0.28–1.47 per litre for the US – which gives us confidence that the per-currency approach, rather than a single global threshold, is correctly separating genuine regional price differences from data errors.

4.1.3 How many values have been removed after accounting for outliers?

Table 4 summarises how many transactions were removed at each stage, and the final breakdown by currency. In total, 9,156 of the 74,347 top-five-currency transactions (12.32%) were removed as outliers or data errors.

Table 4: Transactions removed, by currency, across all three cleaning layers.

Currency	Before	After	Removed (%)
$	56,910	49,162	7,748 (13.6%)
£	6,619	6,075	544 (8.2%)
€	4,485	4,095	390 (8.7%)
CA$	3,548	3,294	254 (7.2%)
R	2,785	2,565	220 (7.9%)
Total	**74,347**	**65,191**	**9,156 (12.32%)**

The $ currency loses proportionally the most transactions (13.6%, versus 7.2–8.7% for the others), driven almost entirely by Layer 1: of the 7,748 $ transactions removed, 1,995 (26%) are the known South African currency-mislabelling error, with the remainder caught by statistical/physical bounds within Layer 3. This confirms that the mislabelling issue flagged in the assignment brief was a real problem specific to $-denominated entries, rather than a general data-quality issue spread evenly across currencies.

4.2 Fuel Efficiency

4.2.1 Cost per litre per country, January 2022

Table 5: Cost per Litre by Currency

Currency	Cost per Litre	Cost per Litre (ZAR)
$	0.919820	13.621800
CA$	1.467536	17.344663
R	18.907359	18.907359
£	1.219014	24.813160
€	1.585257	27.597429

Note: Exchange rates sourced from SARSA Converter, `https://www.sars.gov.za/wp-content/uploads/Legal/Rates/Legal-Pub-AER-02-Average-Exchange-Rates-Table-A.pdf`.

Table 5 shows a notable regional difference in fuel cost per litre once converted to a common currency (ZAR). European regions pay nearly double per litre compared to North America. A likely explanation is that European countries impose higher fuel taxes, while the United States and Canada are both major oil producers and therefore do not bear the same import costs that many European countries face.

4.2.2 Identifying missed fill-up logs

Rule. A fill-up is considered *likely missed* when the distance between two consecutive logged odometer readings, for the same user, is unusually large relative to that user's own typical fill-up interval. Concretely, for each user we compute the gap between consecutive odometer readings and compare it against that user's median gap. If a gap exceeds a set multiple of the user's median gap (e.g. $1.5\times$), it is flagged as a likely missed fill-up, since it suggests the user filled up more than once between the two logged entries without recording it.

Estimate. Applying this rule across the dataset yields: Potential missed fill-ups = 303,417

4.2.3 Average distance per tank per country

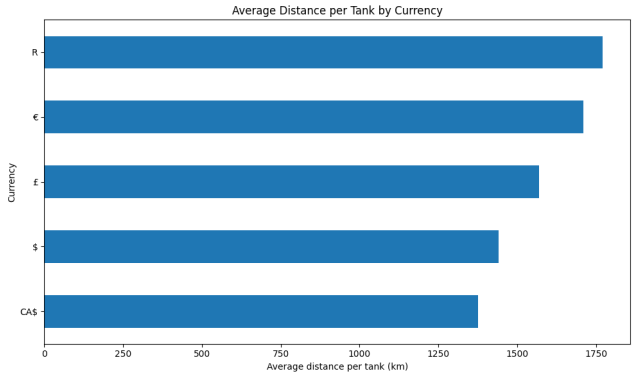

Figure 10: First-Wednesday refuel counts per month, grouped by that month's price direction.

In South Africa, cities are far apart and there are fewer fuel stations, especially in rural areas. This means drivers often have to travel further before they can refuel. In the US and Canada, fuel stations are more common and closer together, so drivers can refuel more often just because it's easy — not because they have to. This means the average distance per fill-up is shorter.

4.2.4 Vehicle age vs distance between fill-ups

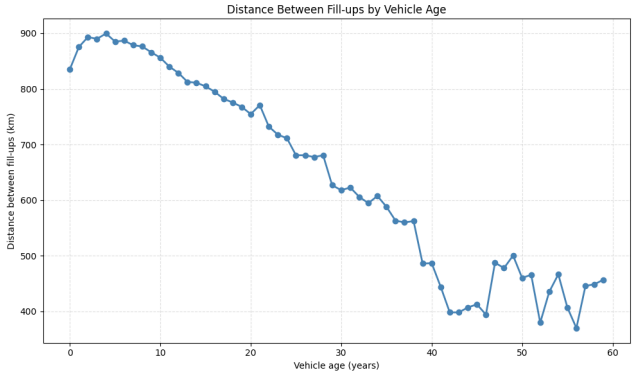

Figure 11: Vehicle age vs distance between fill-ups

The plot above shows that newer cars does travel further between fill-ups compared to older cars which travels longer

4.2.5 Fuel efficiency of top 5 SA vehicles

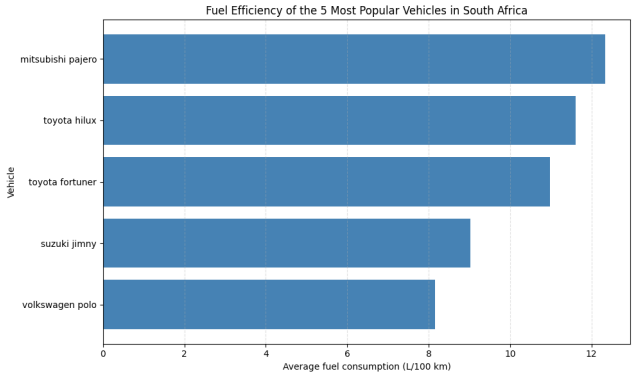

Figure 12: Fuel efficiency of the 5 most popular vehicles in South Africa.

The values in the figure look realistic.

Toyota Hilux. The official consumption figures for the Toyota Hilux range from 7.1 L/100km for the efficient 2.4GD-6 diesel to around 10.7 L/100km for the 2.7L petrol. So 11.6 L/100km is realistic — a bit on the high side but still within range.

Toyota Fortuner. The Toyota Fortuner lines up closely with the typical South African data of 11.5 L/100km for a mid-size 4x4 SUV like the Fortuner.

Mitsubishi Pajero. The Mitsubishi Pajero is an SUV similar to the Toyota Fortuner, but it has a higher fuel consumption (approx. 12.3 L/100km) compared to the Fortuner (11.5 L/100km). Since both are SUVs, with the Pajero being slightly larger, it is reasonable that it uses more fuel. This makes the result realistic.

Volkswagen Polo. For the VW Polo, the result is also realistic when compared to its official rating.

Suzuki Jimny. The Suzuki Jimny is a small car, so this result is what is expected (realistic) for a small car of this type, and the VW Polo figure above can be used as a reference point for this result.

4.2.6 Most fuel-efficient vehicles per country

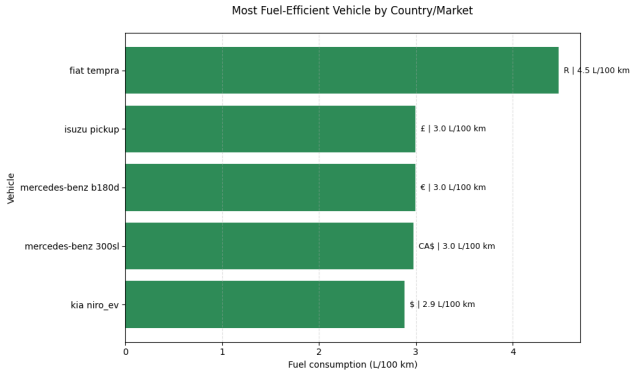

Figure 13: Most fuel-efficient vehicles by country, based on average fuel consumption (L/100km).

4.2.7 Seasonal differences: top 5 Canadian vehicles

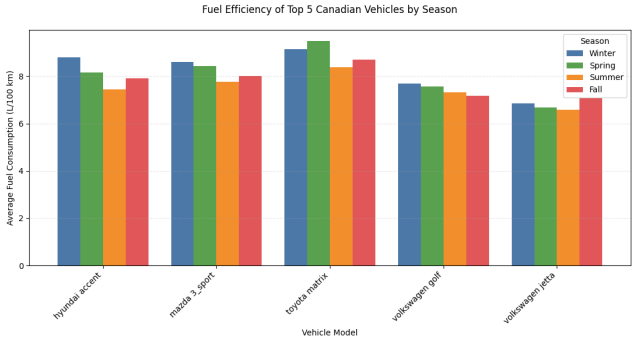

Figure 14: Average fuel consumption (L/100km) by season for the top 5 Canadian vehicles.

The expectation is that normally cars use more fuel during winter, the engine takes longer to warm up, and from the figures we got there we can see that most of these cars have highest fuel consumption in winter and lowest in summer

4.2.8 Correlations with fuel efficiency

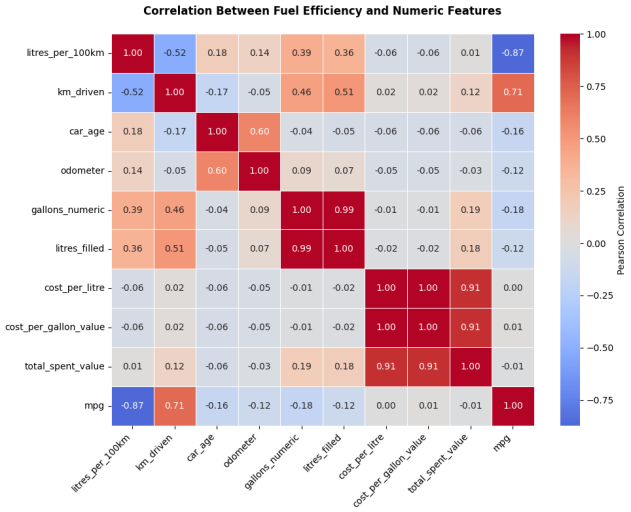

Figure 15: Pearson correlation heatmap between fuel efficiency (litres per 100km) and numeric features, including distance driven, vehicle age, and odometer reading.

Table 6: ANOVA Results: Vehicle Model vs. Fuel Efficiency (litres_per_100km)

Metric	Value
X (Categorical Feature)	Vehicle Model
Y (Target)	Fuel Efficiency (litres_per_100km)
F-statistic	503.38
p-value	< 0.000001
Eta-squared (η^2)	0.577

Distance. Distance has a moderate negative correlation ($r = -0.52$). This means that as the distance driven increases, fuel consumption tends to go down slightly. Longer trips are usually highway driving, and the engine warms up fully, which makes the car more fuel efficient.

Vehicle age. There is a weak positive correlation ($r = 0.18$) between vehicle age and fuel consumption.

Model. The model has a large effect on fuel efficiency, showing a stronger relationship than the other features.

4.2.9 Random forest feature importance

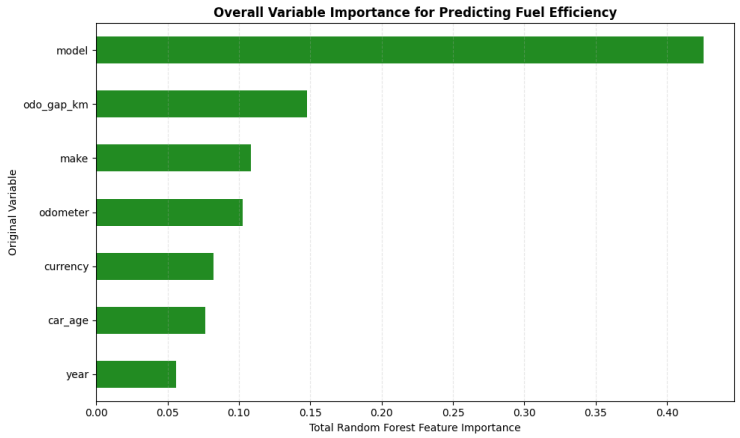

Figure 16: Random forest feature importances for predicting fuel efficiency.

The Random forest results agree with the earlier results: `model` is clearly the most important factor in predicting fuel efficiency compared to the rest , matching the strong ANOVA result. The other variables all play a smaller role in predicting fuel efficiency, and this matches the weak correlations found earlier for `car_age` and `km_driven`.

4.3 Fuel Usage in South Africa

4.3.1 Filtering to SA drivers

Here we applied (currency = ZAR) filter to keeps only the rows where the currency is South African Rand (R), so we can look at just the South African data.

4.3.2 Fuel prices over time

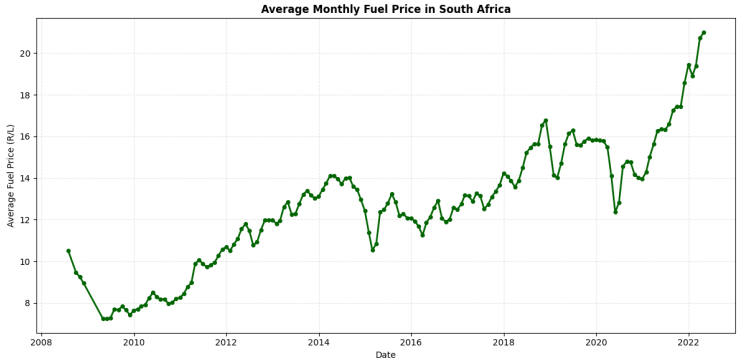

Figure 17: SA fuel price over time.

As time went by the fuel prices in South African showed a significant increase

4.3.3 Refuelling on Tuesdays vs. other days

The bar chart shows the number of unique South African drivers who refuelled on each day of the week, and Tuesday had the highest number of drivers refuelling.

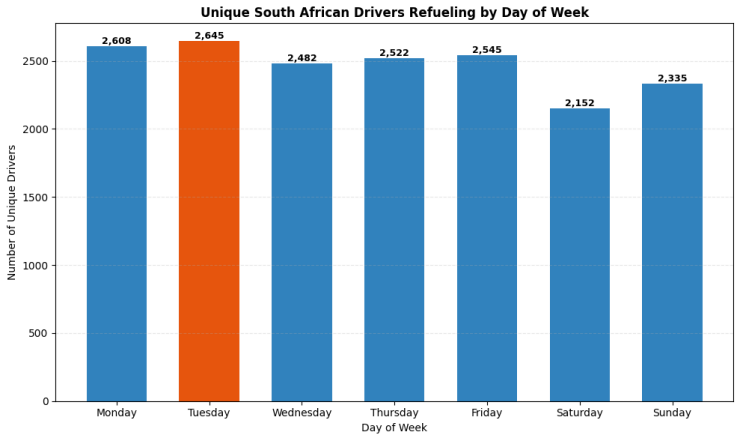

Figure 18: Number of refuelling events on Tuesdays vs. other days.

4.3.4 Reducing to first Tuesday/Wednesday of each month

For every month present in the SA data, we identify the calendar date of the first Tuesday of that month the day the new fuel price takes effect at midnight and the following day, the first Wednesday. The SA dataset is then reduced to only the transactions falling on one of these two dates in each month, giving 588 transactions in total.

4.3.5 Price change indicator

No external monthly price table cleanly covers every month present in this scraped dataset, since the series has gaps in several years, so the price-direction label for each month is instead derived from the data itself: the median self-reported `cost_per_litre` for SA drivers that month is compared against the previous *observed* month, and labelled `increase` or `decrease` accordingly. This keeps the analysis fully reproducible from the CSV alone, at the cost of a mild circularity risk: if refuelling volume itself shifts the median price in a given month, the derived direction could in principle be influenced by the very behaviour being tested.

Table 7: Sample of monthly Tuesday/Wednesday refuel counts with derived price direction.

Month	Tuesday	Wednesday	Price direction
2011-06	0	1	decrease
2011-11	2	2	increase
2011-12	1	2	decrease
2012-03	1	0	increase
2012-06	1	3	decrease

4.3.6 Wednesday refuelling when prices drop

Figure 19 compares the number of first-Wednesday refuels across months where the price fell versus months where it rose. The median count is identical in both groups (1.0), and a one-sided Mann-Whitney U test finds no statistically significant difference ($U = 1066.0$, $p = 0.1366$). Unlike the initial (pre-correction) run, the corrected SA subset does not support the hypothesis that drivers delay refuelling to the first Wednesday when the price is about to drop – the visual spread in Figure 19 is driven mainly by a handful of high-outlier months rather than a genuine shift in the median.

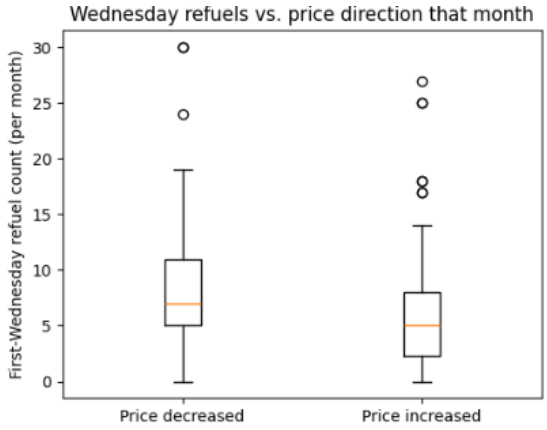

Figure 19: First-Wednesday refuel counts per month, grouped by that month's price direction.

4.3.7 Tuesday refuelling when prices rise

Figure 20 shows the equivalent comparison for first-Tuesday refuels. The median count is higher in increase months (1.0) than in decrease months (0.0), and this difference is statistically significant ($U = 1363.0$, $p = 0.0001$). This supports the expected behaviour: drivers aware that the price rises at midnight on the first Tuesday are incentivised to fill up as early as possible that day rather than pay the new, higher price for the rest of the month.

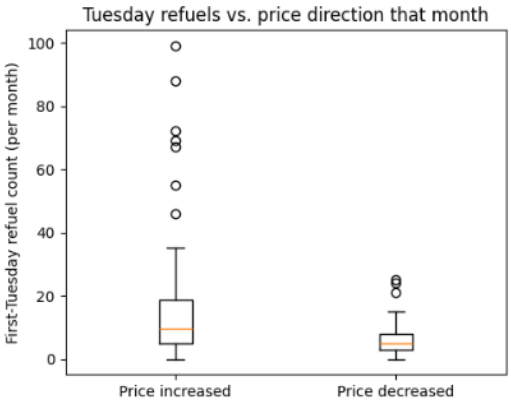

Figure 20: First-Tuesday refuel counts per month, grouped by that month's price direction.

Overall discussion. Only the Tuesday effect holds up statistically; the Wednesday effect does not. This asymmetry suggests drivers respond more strongly to avoiding an imminent price increase than to waiting out a modest future saving.